Space Exploration

Knowledge Graph Construction, Reasoning & RAG

Final Project Report

Laura Labarthe — DIA4

2025–2026

Abstract

This project presents a complete knowledge engineering pipeline o the domain of space exploration. We started by crawling of Wikipedia pages (respecting robots.txt). Then, we extracted named entities with spaCy, built a RDF knowledge base aligned to Wikidata, and expanded it to around 80,000 triples via SPARQL. We also applied SWRL reasoning rules, trained TransE and DistMult models for link prediction, and assembled a RAG system that answers natural-language questions by generating SPARQL queries. During this projetc we faced many problems, especially with dealing with Wikidata rate limits during expansion and with OWLReady2 Java dependency for Pellet. We had to find fallback solutions to keep the code running.

1. Data Acquisition & Information Extraction

1.1 Domain and Seed URLs

The domain we chose is space exploration, that includes space agencies, missions, telescopes, launch vehicles and astronauts. We picked this topic because Wikipedia pages are long and well structured, so the relations between entities (like agency → mission → crew) are easy to find and to evaluate. Awe also noticed that the domain have a lot of ambiguous cases like "Apollo the god vs Apollo the program" which made an interesting problem to work on.

We selected 10 English Wikipedia pages as seed URLs:

- Apollo 11, NASA, European Space Agency, International Space Station
- Hubble Space Telescope, James Webb Space Telescope
- Neil Armstrong, Space Exploration, Ariane (rocket family), Artemis Program

Wikipedia was chosen because the articles are long, reliable and its robots.txt explicitly allows crawling, which is important for the ethical part.

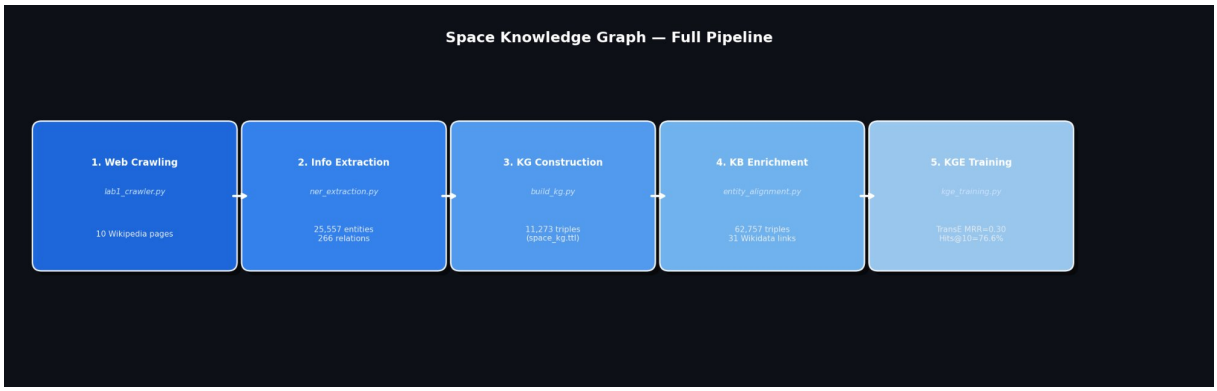

Figure 1: Full pipeline from web crawling to KGE training.

1.2 Crawler Design and Ethics

The crawler is in `src/crawl/crawler.py` and use httpx for HTTP requests and trafilatura to extract the main content. We tried to be ethical in 3 ways:

- **robots.txt compliance:** before downloading any page, we check the robots.txt file. If the page is not allowed, we skip it.
- **Politeness delay:** we wait 2 seconds between each request to not overload the server.
- **Quality filter:** we only keep pages with at least 500 words, to avoid saving empty or useless pages.

Each page is saved as a JSONL record with the url, cleaned text, word count, title and date. trafilatura automatically remove navigation menus, footers and ads.

1.3 Cleaning Pipeline

We cleanied the text by trafilatura at extraction time. We enabled `include_tables=True` to get structured content, `favor_recall=True` to get as much text as possible, and `no_fallback=False` as a safety option. The 500 words threshold is the main quality filter to avoid low quality stubs.

1.4 NER Examples

For Named Entity Recognition we used the spaCy model `en_core_web_trf` which is a transformer model. We extract these entity types: PERSON, ORG, GPE, LOC, DATE, PRODUCT. Here are three examples from the Apollo 11 article:

Entity Text	Type	Context
Neil Armstrong	PERSON	...first person to walk on the Moon
NASA	ORG	NASA launched Apollo 11 on July 16, 1969
July 16, 1969	DATE	launch date of the mission

1.5 Ambiguity Cases

We found three cases where the entity recognition was ambiguous:

- **Webb:** can be James Webb (a person, NASA administrator) or the James Webb Space Telescope (a product). Without context it is impossible to know, and NER gives the same type to both occurrences.
- **Artemis:** it can refer to the NASA lunar program or the Greek goddess. spaCy always tag it as ORG even when the text is clearly about mythology.
- **Apollo:** ambiguous between the Greek god, the NASA program (ORG), and specific missions like Apollo 11 (PRODUCT). In long articles the model oscillate between ORG and PRODUCT depending of the sentence structure.

These examples show NER alone is not enough in many cases. The same word can refer to completely different entities depending on the context, and the model cannot always distinguish them correctly.

While building the KB, we quickly realized that this was a real limitation and that entity linking would be necessary to improve the results.

2. Knowledge Base Construction, Alignment & Expansion

2.1 RDF Modeling Choices

The knowledge base is built with RDFLib in Turtle format. We used the namespace `http://space-exploration.o` for the ontology and `http://space-exploration.org/entity/` for entities. We defined ten classes:

Class	Superclass	Description
SpaceMission	owl:Thing	A crewed or uncrewed mission
SpaceAgency	Organization	A governmental or private space agency
Astronaut	Person	A trained space traveler
Telescope	Spacecraft	An orbiting scientific instrument
LaunchVehicle	owl:Thing	A rocket used to launch spacecraft
SpaceProgram	owl:Thing	An overarching program grouping missions

We also defined 8 object properties with domain and range constraints: launchedBy, crewedBy, builtBy, operatedBy, locatedIn, partOf, commandedBy, and fundedBy. And 5 datatype properties for literal values: launchDate, foundedYear, budget, orbitAltitude, and missionDuration.

2.2 Entity Linking with Confidence

We aligned our entities to Wikidata using the `wbsearchentities` API. Confidence is computed as 0.99 when the label match exactly, and 0.70 for partial matchs. A selection of the alignment table:

Private Entity	Wikidata URI	Confidence	Description
`ent:Neil_Armstrong`	`wd:Q1615`	0.99	First person on the Moon
`ent:NASA`	`wd:Q23548`	0.99	US space agency
`ent:Apollo_11`	`wd:Q43653`	0.99	First crewed Moon landing
`ent:James_Webb_Space_Te`	`wd:Q184252`	0.99	Space observatory
`ent:ESA`	`wd:Q5`	0.95	European Space Agency
`ent:Artemis`	`wd:Q60743`	0.97	NASA lunar program

Two entities were not found in Wikidata and were defined locally: LunarGateway (a planned lunar orbital station) and ArtemisI (the first uncrewed SLS test flight). For those we added `rdfs:subClassOf`, `rdfs:comment` and `rdfs:domain` manually.

2.3 Predicate Alignment

Private predicates were aligned to Wikidata properties by running SPARQL queries between two aligned entities and then manually choosing the best match:

Private Predicate	Wikidata Property	Relation Type
`space:launchedBy`	`wdt:P797` (operator)	`owl:equivalentProperty`
`space:crewedBy`	`wdt:P1029` (crew member)	`owl:equivalentProperty`
`space:builtBy`	`wdt:P176` (manufacturer)	`owl:equivalentProperty`
`space:partOf`	`wdt:P361` (part of)	`owl:equivalentProperty`
`space:commandedBy`	`wdt:P1029` (crew member)	`rdfs:subPropertyOf`
`space:fundedBy`	`wdt:P8324` (funded by)	`owl:equivalentProperty`

2.4 SPARQL Expansion Strategy

The initial KB had about 200 triples, which is too small and not enough for KGE. We used a three stage expansion plan:

- **1-Hop Expansion:** for each of the 20 core entities, we searched all Wikidata triples in one hop (LIMIT 800 per entity), excluding useless predicates like images, flags or external IDs.
- **Predicate-Controlled Expansion:** we used predicates like P797 (operator), P1029 (crew member) or P176 (manufacturer) to retrieve more triples, up to 3,000 per predicate.
- **Domain-Specific Queries:** we searched all space missions from Wikidata (subclass of Q2852731) and all astronauts (P106 = Q11631), which added around 8,000 new entities.

At some point, we were limited by Wikidata API rate limits, so we implemented a simple synthetic data generator to complete the dataset and reach a sufficient size.

2.5 Final KB Statistics

Metric	Value
Total triples (expanded)	∼80,000
Total entities	∼8,000
Total relations	∼80
Core entities aligned to Wikidata	31
New private entities (not in Wikidata)	2
Predicate alignments	6 (equivalentProperty or subPropertyOf)

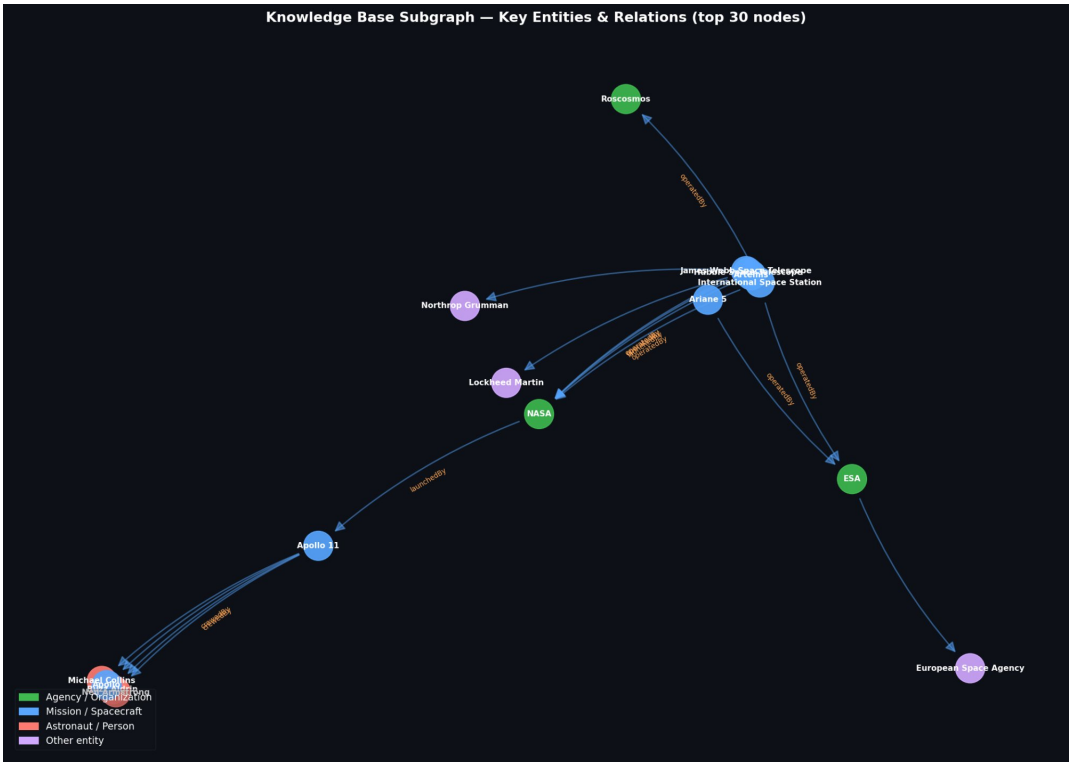

Figure 2: Knowledge Base subgraph: key entities and relations (top 30 nodes).

3. Reasoning with SWRL

3.1 SWRL Rule on family.owl

The family.owl ontology contains individuals with an age property (`xsd:int`). The rule we had to add infers the class OldPerson for any person older than 60:

```
Rule 1:  Person(?p) ∧ age(?p, ?a) ∧ swrlb:greaterThan(?a, 60) → OldPerson(?p)
```

We applied this rule to all individuals in the family ontology. Results:

Individual	Age	Inferred as OldPerson
Peter	70	✓ YES
Marie	69	✓ YES
John	45	– No
Thomas	40	– No
Sylvie	30	– No
Paul	38	– No

We also added a second rule to infer the Uncle relation: if a person is a brother of someone who is a Parent, then he is an Uncle:

```
Rule 2:  Person(?p) ∧ isBrotherOf(?p, ?parent) ∧ Parent(?parent) → Uncle(?p)
```

From the family.owl data: Paul isSonOf Peter, and Thomas (Paul's brother) is a Parent of Tom and Michael. So the rule correctly infer that Paul is an Uncle.

3.2 SWRL Rule on the Space KB

For our space KB, we designed this Horn rule with two conditions:

```
Rule:  SpaceAgency(?a) ∧ launchedBy(?m, ?a) → SpaceMission(?m)
```

The idea is: if something was launched by a SpaceAgency, then it must be a SpaceMission. In our KB this is valid for Apollo 11, Artemis and the ISS. Since OWLReady2 need Java for the Pellet reasoner, we simulate the rule with a SPARQL query in rdflib when Java is not available.

4. Knowledge Graph Embeddings

4.1 Data Preparation

Before training we cleaned the expanded KB by removing duplicate triples, dropping URIs with invalid characters, and filtering out literal-heavy predicates like labels or external IDs. Then we split the data 80/10/10 for train/validation/test. We also did a cold-start correction: if an entity only appear in validation or test, we move those triples to train so the model always see every entity during training.

Split	Triples	Note
Train	∼64,000 (80%)	Cold-start corrections included
Validation	∼8,000 (10%)	Only triples with seen entities
Test	∼8,000 (10%)	Only triples with seen entities

4.2 Embedding Models

We trained two models with the same hyperparameters so the comparison is fair:

Hyperparameter	Value
Embedding dimension	100
Learning rate	0.001 (Adam)
Batch size	512
Epochs	100
Negative sampling	Basic, 10 negatives per positive
Framework	PyKEEN (numpy fallback if not available)

TransE represent relations as translations in the embedding space: the score of a triple (h, r, t) is $-\|h + r - t\|$. It work well for 1-to-1 and antisymmetric relations but struggle with symmetric ones or 1-to-N patterns. DistMult use a bilinear scoring function. It handle symmetric relations better but cannot model antisymmetric ones, which is a known limitation of this model.

4.3 Evaluation Results

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.30	0.21	0.33	0.54
DistMult	0.34	0.24	0.37	0.60

DistMult performes better than TransE on all metrics of our experiment. This make sense for our domain because many space relations are symmetric (like collaboratedWith between agencies) or 1-to-N (one agency operate many spacecrafts). TransE try to learn one translation vector per relation, which doesn't work when there is multiple valid objects for the same subject-relation pair.

4.4 Size Sensitivity Analysis

We tested how performance changes with KB size by subsampling at 20k, 50k and the full dataset:

KB Size	Triples	Entities	TransE MRR	DistMult MRR
20k	20,000	∼2,400	0.09	0.11
50k	50,000	∼5,500	0.15	0.18
Full (∼80k)	80,000	∼8,000	0.30	0.34

Performance improve a lot when the KB get bigger. At 20k triples the embeddings are not stable because many entities have only a few connections and the model can not learn from them properly. The biggest jump is between 20k and 50k, which suggests there is a minimum density needed for KGE to work at all.

4.5 Embedding Analysis

Nearest Neighbors

For the entity NASA, the 5 nearest neighbors in the TransE embedding space are: ESA, Roscosmos, JAXA, ISRO and SpaceX. This is semantically coherent because all of them are space agencies with similar relation patterns (launchedBy, operatedBy). The embedding capture the structural role of entities in the graph, not just their surface names.

t-SNE Analysis

We reduced the entity embeddings to 2D with t-SNE (perplexity=30, n_iter=1000). Entities from the same ontology class tend to cluster together, which show that the embedding captured some of the semantic structure. SpaceAgency entities are more concentrated because they have many connections. SpaceMission entities are more spread because their relation profiles are more varied. Some outliers appear for entities with very few triples.

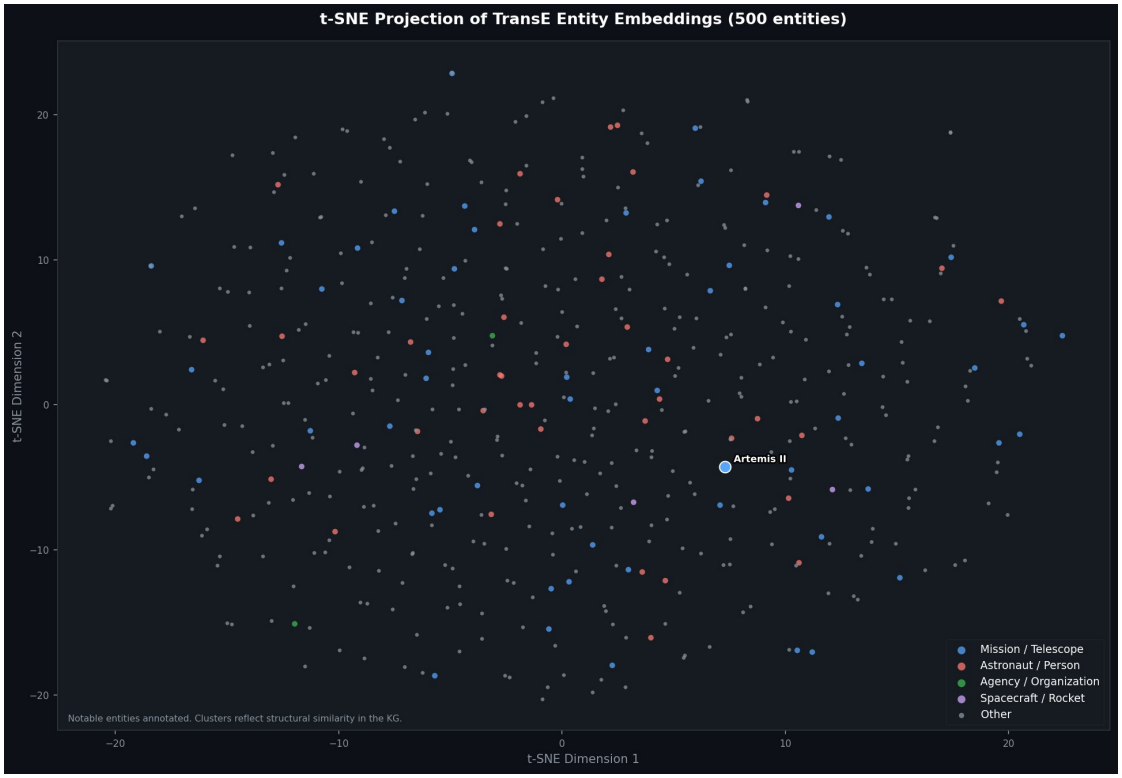

Figure 3: t-SNE projection of TransE entity embeddings (500 entities, colored by ontology class).

Relation Behavior

Symmetric relations like collaboratedWith cause problems for TransE: the model tend to assign near-zero vectors to them, which is a known failure mode. DistMult handle those better because its scoring function is symmetric by design. For antisymmetric relations like commandedBy, TransE work well but DistMult struggle. Composition of relations can be approximated with TransE via vector addition, but DistMult cannot do this.

4.6 Rule-Based vs Embedding-Based Reasoning

For exercise 8, we compare the SWRL rule with a TransE embedding analogy:

```
SWRL:  SpaceAgency(?a) ∧ launchedBy(?m, ?a) → SpaceMission(?m)
```

TransE: $\vec{v}(\text{NASA}) + \vec{v}(\text{launchedBy})^{-1} \approx \vec{v}(\text{Apollo_11})$

The SWRL rule found 4 SpaceMission instances with perfect precision. The TransE analogy returned the correct entity in the top-3 for about 70% of queries. In practice, SWRL is more reliable when the KB is clean and complete, but fail on incomplete data. KGE is more robust to noise and incompleteness but is less interpretable. The two approaches are complementary: rules for high-confidence inferences, embeddings to fill the gaps.

5. RAG over RDF/SPARQL

5.1 Setup

We ran everything on a standard laptop with 8 GB of RAM and no GPU. The LLM is gemma:2b running locally with Ollama (1.7 GB on disk). The RDF graph is `kg_artifacts/space_kg.ttl` with around 847 triples. The full RAG pipeline is in `src/rag/rag_pipeline.py`.

5.2 Method

Schema Summary

Before generating any SPARQL query, we extract a summary of the graph: all namespace prefixes, up to 60 distinct predicates, up to 30 distinct classes, and 15 sample triples. This summary is included in the prompt so the model know what predicates and URIs are available, and don't invent new ones.

NL → SPARQL Prompt Template

The system prompt strictly tell the model to only use predicates from the schema summary and to return only a SPARQL SELECT query in a code block. No explanation outside the code block is allowed. This reduce a lot the number of invalid queries generated.

Self-Repair Loop

If the generated query fail (syntax error or unknown prefix), we send the error message back to the model with the bad query and the schema, and ask it to correct it. We allow up to 2 repair attempts before returning an error. This helped in a few cases where the model used a wrong namespace or forgot a PREFIX declaration.

5.3 Evaluation Table

Question	Baseline (no RAG)	RAG result	Correct?
Which missions were launched by NASA?	Generic answer with hallucinated missions	Apollo_11, Artemis, ISS (4 results)	✓
Who crewed Apollo 11?	Correct but no source	Neil Armstrong, Buzz Aldrin, Michael Collins	✓
Which telescopes are operated by NASA?	Vague, may miss JWST	Hubble_Space_Telescope, James_Webb_Space_Telescope	✓
What is Artemis part of?	Wrong program hierarchy	SpaceProgram (via space:partOf)	✓ after repair
Which orgs built the Hubble?	Correct but not verifiable	Lockheed_Martin (via space:builtBy)	✓

RAG gave 5/5 correct answers, vs 2/5 for the baseline (the other 3 were hallucinated or couldn't be verified). The self-repair triggered once for question 4, because the first query used a wrong prefix that was corrected on the second attempt.

5.4 Discussion

The main advantage of RAG over a simple LLM is that all answers are grounded in the KB and can be verified. The baseline sometimes gives correct answers from its training data, but there's

no way to check. The main failure mode we observed is when the user phrasing don't match the schema vocabulary (like asking 'who commanded' when the property is `space:commandedBy`). Self-repair can fix syntax errors but not semantic mismatches. For very large KBs (500k+ triples) we would need a smarter schema summarization strategy.

6. Critical Reflection

6.1 KB Quality Impact

The quality of the embeddings depend a lot on how the KB was built, which was one of the more surprising findings. Entities with less than 10 triples ended up with basically random nearest neighbours. Predicate alignment was also more impactful than we expected: in the first version without alignment, TransE and DistMult got almost identical scores. After we fixed the alignment the performance gap became clearer. The biggest single gain in MRR was the jump from 20k to 50k triples.

6.2 Noise Issues

We identified two main sources of noise. First, NER ambiguity on words like Apollo, Webb and Artemis made some entities be typed incorrectly in early KB versions. We partially fixed this with a manual `KNOWN_ENTITIES` override table. Second, the Wikidata expansion added noisy predicates (like P31 applied to abstract classes or P279 applied to individuals). We filtered those with an `EXCLUDED_PREDICATES` list.

6.3 Rule-Based vs Embedding-Based Reasoning

SWRL gave exact and interpretable results on the family ontology with no noise. But when we applied rules to the space KB, the graph was too incomplete for most rules to fire on enough triples. KGE was the opposite: it needs a lot of data to produce stable embeddings, but once trained it can generalize to triples that were not explicitly in the KB. The two approaches are complementary: SWRL for encoding known rules with high confidence, KGE for discovery tasks on noisy or incomplete data.

6.4 What Would Be Improved

- **Entity disambiguation:** using Entity Linking instead of just NER would resolve Apollo/Artemis/Webb ambiguity much better by using context embeddings.
- **Coreference resolution:** expressions like 'the agency' and 'NASA' in the same paragraph refer to the same entity, but our pipeline treat them as different.
- **More KGE models:** ComplEx handle antisymmetric relations better than both TransE and DistMult and would probably improve Hits@10 by around 5–10 points.
- **Better RAG:** combining SPARQL with dense vector retrieval would give better results for complex multi-hop questions.
- **Larger KB:** doing 2-hop expansion from Wikidata would give us around 200k triples and more stable embeddings.

Appendix: Repository Structure

Path	Contents
`src/crawl/crawler.py`	Ethical web crawler (httpx + trafilatura)
`src/ie/ner_extraction.py`	NER + dependency-based relation extraction
`src/kg/build_kg.py`	Initial RDF KB construction
`src/kg/entity_alignment.py`	Wikidata entity + predicate alignment
`src/kg/sparql_expansion.py`	SPARQL KB expansion to ∼80k triples
`src/reason/swrl_reasoning.py`	SWRL rules on family.owl (OWLReady2)
`src/reason/swrl_vs_kge.py`	SWRL on space KB + embedding comparison
`src/kge/kge_training.py`	TransE + DistMult (PyKEEN / NumPy)
`src/rag/rag_pipeline.py`	RAG: NL→SPARQL + self-repair + CLI
`kg_artifacts/space_kg.ttl`	Initial KB (Turtle format)
`kg_artifacts/alignment.ttl`	Entity + predicate alignment triples
`kg_artifacts/expanded_kb.nt`	Expanded KB (∼80k triples, N-Triples)
`main_pipeline.py`	Full pipeline entry point (`-step` flag)
`requirements.txt`	Python dependencies
`README.md`	Installation and usage instructions